A continued-pretraining trainer that proves its own resumes: design of PinoAI's CPT trainer v2 and the results of its GPU plumbing test

PinoAI engineering note · ENG-005 · technical report · Taglish edition: ENG-005-cpt-trainer-v2-plumbing.tl.qmd

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-10-06

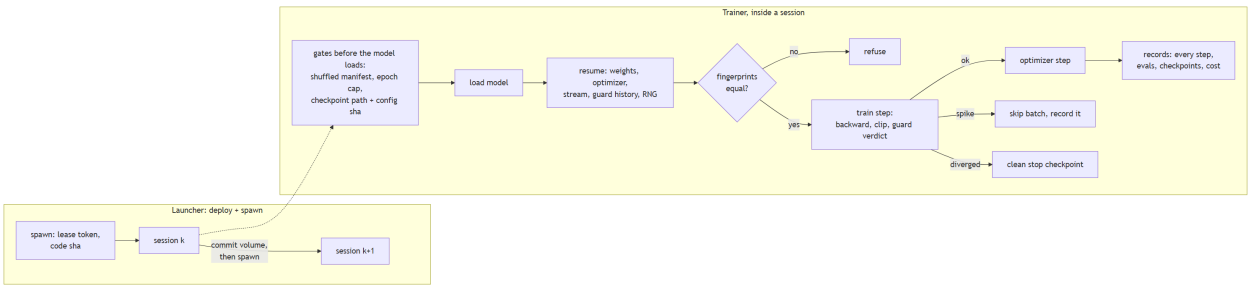
Abstract

Before PinoAI spends about $750 of GPU time on its first continued-pretraining (CPT) run, the trainer has to show, on the real GPU and the real model, that it can stop and resume without changing the run. This note describes the trainer's design and reports its plumbing test: a short full-parameter run of Gemma 4 E4B on one H100, with forced pauses and resumes, plus a 10-step LoRA run.

1. **Resumes are exact where they can be, and checked where they can't.** After a resume from another run's checkpoint, step 40 matched the original run bit for bit (loss 2.057240415364504, grad norm 50.25), and all 8 components of the 8-bit optimizer state fingerprint were equal. A forced pause at step 48 moved the next loss by 0.0156, against a bound of 0.4505 fixed before the number was computed.
2. **The LoRA path works on the GPU.** It paused at step 5, resumed with equal fingerprints, and finished at step 10. It ran at 4,242–4,249 tokens/s, within 1.7% of ENG-003's rate for the same settings.
3. **Full-parameter training was faster than registered.** 5,659–5,729 tokens/s against a band of 4,041–5,468, so the throughput criterion is **not met, in the fast direction**. It is reported, not explained away.
4. **The held-out curve has a stability gap.** Tagalog bits per byte rose from the base 0.8850 to 0.9307 after 10.5M tokens, then fell to 0.8455 at 30.4M, below the base.
5. **It took ten amendments' worth of defects.** Two were in the 8-bit optimizer path that no CPU test reaches, one in a guard, and seven in the launcher and the reporting around the trainer. Each fix shipped with a control that fails on the old code (PM-017).

1. What the trainer has to guarantee

A CPT run of 2×10^9 tokens on one H100 takes about 117 hours (ENG-003). Modal functions live at most 24 hours, so the run is a chain of about six sessions. Each one resumes from the last checkpoint. If a resume silently changes the optimizer state, the data order or the learning-rate schedule, the run is a different experiment from the one that was registered, and nothing in the loss curve will say so. The trainer therefore has to make every resume **checkable**, not just possible.



The CPT trainer v2 and its launcher. Every arrow that crosses a session boundary carries a check.

The guarantees, each tied to the incident or review item that made it necessary:

Table 1: The trainer's guarantees.

Guarantee	How	Why
A bad corpus or a wrong checkpoint costs seconds, not a GPU session	shuffled-manifest gate, epoch-cap projection, checkpoint path and config hash, all checked before the model loads	review d457e2 B1/B5; PM-017 defects 5 and 8
A resume restores the same run	weights or adapter, optimizer state, data-stream position, guard history and RNG in one atomic checkpoint; the config hash must match	review d457e2 B4
The resume is verified, not assumed	L2-norm fingerprints of every optimizer-state component, computed from the stored 8-bit codes without dequantising, compared after load; a mismatch refuses	Amendments 8–9
A bad step is never applied	non-finite values and loss spikes STOP before the optimizer step; a grad-norm spike with a normal loss SKIPS the batch, within limits	review d457e2 B2; Amendment 11
A stop loses no applied work	a guard stop writes a clean checkpoint of the last applied step	Amendment 11
Every step is on record	one compact record per step attempt (loss, grad norm, applied), plus the train-loss breakdown by mix bucket and document language from Amendment 16	Amendments 14 and 16
Cost is measured, not inferred	billed seconds include model loading, evaluation and checkpoint writes; peak GPU memory around evaluation and saving	Amendment 14
A run survives the laptop	the session loop runs server-side: a session commits the volume, then spawns the next; a lease admits one writer per run; a stale deploy refuses to train	Amendment 14; PM-017 defects 6 and 9
A run's lineage survives its sessions	run_meta keeps the first session's start and parent; each session appends its own environment	Amendment 17

2. The plumbing test, as registered

DEC-005 registered the test before it ran. The full arm is Gemma 4 E4B at revision 411aa17b with full-parameter training, torchao 8-bit AdamW with bf16 stochastic rounding, sequence length 4,096, micro-batch 4 with gradient checkpointing, 524,288 tokens per step and learning rate 1×10^{-5} with 1% warm-up, on the "pilot4" corpus. It ran with a forced pause at step 48. The LoRA arm is r = 64 on all linear layers with learning rate 2×10^{-4}: 10 steps, with a forced pause at step 5. The pass criteria:

Table 2: Plumbing criteria (DECISION_LOG DEC-005).

Criterion	Rule
Optimizer state survives the checkpoint	every fingerprint component equal to within 10^{-6} relative
A resume does not jump	|first loss after the resume − last loss before| ≤ 3 × the step-to-step noise SD (the SD rule was fixed before computing, below)
No guard stop	the run is not stopped by a guard
Records are written	train, eval and checkpoint records in the lab schema
Throughput	within ±15% of ENG-003′s 4,754.6 tokens/s for the same settings (4,041.4–5,467.8)
LoRA plumbing	the 10-step run completes

The noise SD is the sample SD (ddof = 1) of consecutive loss differences over the up to 50 applied steps before the resume. The logged value was wrong on short histories (PM-017 defect 7), so research fixed this rule in DECISION_LOG before the number existed.

3. Results

3.1 Resumes

Table 3: Resume results.

Resume	What was compared	Result
Across runs, step 39 → 40 (`--init-from` the parent's step-39 checkpoint)	the continuation's step 40 against the parent's step 40	identical: loss 2.057240415364504, grad norm 50.25; 8 of 8 fingerprint components equal
Forced pause, step 48 → 49	the loss jump against 3 × SD (47 differences)	0.0156 against 0.4505: **pass**
LoRA, forced pause, step 5 → 6	the loss jump against 3 × SD (4 differences)	−0.1144 against 0.2433: **pass**, weak evidence (4 differences); 2 of 2 fingerprint components equal

The first row is the strongest result in this note. The two runs ran on different days, in different containers and sessions, and step 40 still came out identical to the last bit. So the weights, the 8-bit optimizer state, the data stream position and the RNG all crossed the checkpoint intact. That is the property the pilot's preemption safety rests on. Bit identity did not hold at step 45: the same batch gave grad norm 318 in the parent run and 42 in the continuation. That is one observation (n = 1) of state-dependent amplification under a 1-step warm-up, not a measured rate (PM-017).

3.2 Throughput

The full arm's logged steady-state steps ran at 5,659–5,729 tokens/s (median 5,697, 5 logged steps, excluding each session's first step). That is **above** the registered band's upper edge of 5,467.8, so the throughput criterion is **not met (high)**. The criterion was two-sided on purpose, because a large deviation either way means the measurement or the setup differs from ENG-003. We report the miss as it is and do not explain it after the fact. The cost estimates for the LR check use the measured rate, with ENG-003′s rate as the slow case.

The LoRA arm ran at 4,242 and 4,249 tokens/s. That is 1.7% and 1.5% below ENG-003′s 4,315 for the same settings (micro-batch 4, checkpointing on), inside its ~4% run-to-run spread. (ENG-003′s 4,953 is LoRA's fastest setting, micro-batch 1 with checkpointing off, and is not the right comparison.)

3.3 The held-out curve

Table 4: Full arm, held-out evaluation (300 documents per bucket). The 30.4M row is from the continuation run.

Tokens trained	Tagalog bits per byte	Out-of-domain literary bits per byte
0 (base)	0.8850	1.4206
10.5M	0.9307	1.6805
20.4M	0.8697	1.4917
30.4M	0.8455	1.4202

The curve first gets worse, then better. Bits per byte on held-out Tagalog rose 5.2% by 10.5M tokens, then fell below the base by 30.4M; the out-of-domain literary text rose 18% and came back to the base. This is the "stability gap" seen when continued pretraining starts on a new distribution. The rule registered for the plumbing (back below the base by 40M tokens) held at 30.4M. The run was stopped at step 58 before the 40M point (PM-017 defect 6), so the 40M value is unobserved. The lesson for the LR check: an arm compared at a single early point can look worse than it will be. The LR check therefore evaluates at 0, 25M and 50M tokens and flags an arm that is still inside the gap.

Two cautions. Bits per byte from runs with different `eval_max_docs_per_bucket` values are not comparable, because they score different document samples. The LoRA plumbing used 100 documents and reports a base of 0.9332 on a different Tagalog sample. Also, this is one run at one learning rate. It describes this run's trajectory and does not estimate the gap in general.

3.4 Time that is not training

Table 5: Measured non-training time (from the run records).

Item	Measured
model load + setup, first session	69 s (full), 84 s (LoRA)
setup with `--init-from` (includes hashing the 28.9 GiB parent checkpoint)	146 s
setup on a warm resume	9–14 s
one held-out evaluation, 300 documents per bucket	58–73 s
full-arm checkpoint write + hash (one checkpoint, from record timestamps)	~92 s, 28.91 GiB
LoRA checkpoint	1.91 GiB

Amendment 14 now records these directly (`save_s`, `hash_s`, `eval_s`, peak memory, and billed seconds on the final status), so the pilot's cost is measured rather than reconstructed from timestamps.

4. What it took

The plumbing test did its job by failing. The defects it surfaced are documented in PM-017 and the fixes are DEC-005 amendments, each registered with its hash and a control that fails on the code before it:

Table 6: Trainer amendments after the first GPU run. The LR check runs on A14 and the pilot on A17.

Amendment	Change	Trainer sha256 (prefix)
A8	write the learning rate in place when torchao keeps it as a Tensor	2004c9e3
A9	fingerprint 8-bit optimizer states from their stored codes, chunked, never dequantised	5d4d5235
A11	a grad-norm spike with a normal loss skips the batch; stop limits; clean stop checkpoint; `--init-from`	e702a6ab
A14	noise-SD rule in code; checkpoint checks before the model loads; the every-step record; cost and memory records; server-side session loop with lease + code hash; byte-exact copy-back	43ac6123
A16	train loss by mix bucket and by document language per step (detached per-token NLL from the same logits)	c38b6c4e
A17	run_meta keeps its parent and start across sessions	87b4700f

Two process results matter beyond the code. First, the launcher needed controls of its own. The most dangerous defect, a server-side chain that would have resumed from a stale checkpoint, was in the launcher and was caught by a CPU control before any GPU use. Second, claims about cloud behaviour ("detached runs survive a laptop sleep", "the next session sees the last checkpoint") needed a reproducing control. Both claims were wrong as first stated.

5. Cost

The plumbing line was approved at $15. Its spend was about $12.7 (estimates from the run records at $3.95 per H100 hour; Modal billing is authoritative). That covers three full-arm runs (the failures that produced amendments 8, 9 and 11), the step-2 checkpoint smoke test, the failed continuation launch, the continuation, and the LoRA run (about $1.55 billed to its step 10, plus its final evaluation).

6. Limitations

- **One run per arm, one learning rate.** Nothing here says how fast E4B learns Filipino or which learning rate is best. That is the LR check's question.
- **The pilot4 corpus, not the registered training corpus.** The curve values describe pilot4's held-out split.
- **Small samples.** Throughput rests on 5 logged steps. The LoRA resume criterion rests on 4 loss differences. The step-45 divergence is n = 1.
- **The server-side chain has not run on a GPU yet.** It was verified on CPU (3 chained sessions; checkpoint visibility 2 of 2 with the fixed order, 0 of 2 with the buggy one). The LR check's 3×10^{-5} arm pauses at step 10 on purpose to exercise it on the GPU.
- **Throughput above the band is unexplained.** The plumbing ran on the pilot4 corpus in containers that may differ from ENG-003's; we did not test causes.

Reproducibility

Table 7: Files.

Item	Where
Run records	`experiments/results/cpt/<run_id>/` for 152638, cont39-210641 and lora-114155: byte-exact copies of the Modal Volume files (compared 2026-10-06)
Numbers in this note	`experiments/reports/_eng005.py` prints and asserts each one
Step-48 guard history	`experiments/results/cpt/cpt-e4b-plumbing-pilot4-cont39-20261005-210641/guard_history_step48.json`, read read-only from the checkpoint
Code and configs	the hashes in DECISION_LOG DEC-005 (Amendments 7, 8, 9, 11, 14, 16, 17)
Defects and fixes	PM-017

Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

AI assistance. Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.